Employee Management System Documentation

Project: `EmployeeApi`
Generated: September 8, 2026
Scope: Backend API structure, authentication, authorization, employee password login, migrations, Dapper stored procedures, SMTP email notifications, and testing.

1. Project Overview

This project is an ASP.NET Core Web API for employee management. The API is used by a frontend or Postman through JSON requests and responses. The backend contains controllers, services, models, view models, EF Core database configuration, Dapper stored procedure calls, JWT authentication, role authorization, and SMTP email sending.

High-Level Request Flow

```
Frontend/Postman
  -> API Controller
  -> Service Interface
  -> Service Implementation
  -> EF Core or Dapper Stored Procedure
  -> SQL Server
  -> JSON response
```

2. Folder Structure

```
EmployeeApi/
  Api/v1/
    AuthApiController.cs
    EmployeeApiController.cs
    DepartmentApiController.cs
    ClientApiController.cs
    AssignTaskApiController.cs

  Configuration/
    MailSettings.cs

  Data/
    ApplicationDbContext.cs
    EmployeeApiDatabaseInitializer.cs

  Documents/
    Employee_Management_System_Documentation (1).pdf
    Employee_Management_System_Full_Documentation.html
    Employee_Management_System_Full_Documentation.pdf

  Migrations/
    ... EF Core migration files ...

  Model/
    AssignTask/AssignedTask.cs
    Client/Client.cs
    Department/Department.cs
    Employee/Employee.cs

  Service/
    AssignTask/
    Client/
    Department/
    Employee/
      IEmployeeService.cs
      EmployeeService.cs
    Notifications/
      IEmployeeEmailSender.cs
      SmtpEmployeeEmailSender.cs

  ViewModel/
    Auth/
    Employee/
    Department/
    Client/
```

```
    AssignTask/

  appsettings.json
  appsettings.Development.json
  EmployeeApi.csproj
  EmployeeApiServiceRegistrar.cs
  Program.cs
```

3. Controllers

Controllers receive HTTP requests and return HTTP responses. In this project, controllers do not render server-side views. They return JSON to the frontend.

Controller	Route	Responsibility
`AuthApiController`	`/api/auth`	Login and JWT token generation.
`EmployeeApiController`	`/api/employee`	Employee list, detail, create, update, delete.
`DepartmentApiController`	`/api/department`	Department CRUD operations.
`ClientApiController`	`/api/client`	Client CRUD operations.
`AssignTaskApiController`	`/api/assign-task`	Assigned task CRUD operations.

4. Models And ViewModels

Model

A model represents the database entity. Example: `Model/Employee/Employee.cs`.

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string PhoneNumber { get; set; } = string.Empty;
    public decimal Salary { get; set; }
    public int DepartmentId { get; set; }
    public int? ClientId { get; set; }

    [JsonIgnore]
    public string PasswordHash { get; set; } = string.Empty;

    public string Role { get; set; } = "Employee";
}
```

ViewModel

A view model represents request or response shape. Example: `CreateEmployeeViewModel` receives a plain password from the frontend, while the database model stores only `PasswordHash`.

```
public class CreateEmployeeViewModel
{
    public string Name { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string? PhoneNumber { get; set; }
    public decimal Salary { get; set; }
    public int? ClientId { get; set; }
    public int DepartmentId { get; set; }
    public string Password { get; set; } = string.Empty;
    public string Role { get; set; } = "Employee";
}
```

5. Dependency Injection

Services are registered in `EmployeeApiServiceRegistrar.cs`. This lets controllers ask for interfaces in constructors.

```
services.AddScoped<IEmployeeService, EmployeeService>();
services.AddScoped<IAssignTaskService, AssignTaskService>();
services.AddScoped<IDepartmentService, DepartmentService>();
services.AddScoped<IClientService, ClientService>();
services.AddScoped<IEmployeeEmailSender, SmtpEmployeeEmailSender>();
```

Meaning:

```
When code asks for IEmployeeService,
ASP.NET Core gives EmployeeService.
```

6. Program.cs Configuration

`Program.cs` configures application startup.

- Registers controllers and JSON options.
- Registers SQL Server DbContext.
- Binds `MailSettings`.
- Registers application services.
- Configures JWT bearer authentication.
- Configures authorization.
- Configures Swagger and CORS.
- Runs pending EF Core migrations on startup.

```
builder.Services.Configure<MailSettings>(
    builder.Configuration.GetSection("MailSettings"));

app.UseAuthentication();
app.UseAuthorization();
```

> **Important:** `UseAuthentication()` must run before `UseAuthorization()`.

7. Authentication And JWT

Authentication answers: who is the user?

Login endpoint:

```
POST /api/auth/login
```

Request body:

```
{
  "username": "admin",
  "password": "admin123"
}
```

For employee login, `username` means employee email:

```
{
  "username": "employee@example.com",
  "password": "EmployeePassword"
}
```

Login Paths

Login Type	How It Works	Role
Temporary config login	Checks `ApiUser` from `appsettings.json`.	`Manager`
Employee database login	Finds employee by email, verifies password hash.	Role from database: `Manager` or `Employee`

JWT Claims

```
new Claim(JwtRegisteredClaimNames.Sub, username)
new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
new Claim(ClaimTypes.Name, username)
new Claim(ClaimTypes.Role, role)
new Claim(ClaimTypes.NameIdentifier, employeeId.ToString())
```

The most important claim for role authorization is:

```
new Claim(ClaimTypes.Role, role)
```

8. Role-Based Authorization

Authorization answers: what can the user access?

The project uses two roles:

- `Manager`
- `Employee`

Manager-Only Example

```
[Authorize(Roles = "Manager")]
```

Manager Or Employee Example

```
[Authorize(Roles = "Manager,Employee")]
```

HTTP Status Meaning

Status	Meaning	Example
`401 Unauthorized`	No valid login/token.	Wrong password, missing token, expired token.
`403 Forbidden`	Logged in but role is not allowed.	Employee token calls Manager-only API.

9. Employee Password Login

Plain passwords are never stored in the database. The frontend sends a plain password only during create and login.

Create Employee Password Flow

```
Frontend sends model.Password
  -> EmployeeApiController receives CreateEmployeeViewModel
  -> PasswordHasher hashes model.Password
  -> employee.PasswordHash stores hash
  -> Employee saved to database
```

Hashing code:

```
var hasher = new PasswordHasher<EmployeeEntity>();
employee.PasswordHash = hasher.HashPassword(employee, model.Password);
```

Login Password Verification

```
var hasher = new PasswordHasher<Employee>();
var passwordResult = hasher.VerifyHashedPassword(
    employee,
    employee.PasswordHash,
    request.Password);
```

If the result is `PasswordVerificationResult.Failed`, login returns `401 Unauthorized`.

10. EF Core Migrations

Migrations are needed when database table structure changes. They were needed for `PasswordHash` and `Role`.

```
dotnet ef database update --project EmployeeApi
```

Check columns in SQL Server:

```
SELECT COLUMN_NAME, DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'Employees';
```

> **No migration is needed** for controller logic, authorization attributes, Dapper method calls, stored procedures created manually in SSMS, or SMTP configuration.

11. Dapper And Stored Procedures

Dapper is used to call stored procedures directly from the service layer.

Package:

```
<PackageReference Include="Dapper" Version="2.1.79" />
```

Connection String

```
_connString = configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException("Connection string not found.");
```

Implemented Stored Procedure Methods In EmployeeService

Service Method	Stored Procedure	Purpose
`GetByEmail`	`[dbo].[GetEmployeeByEmail]`	Employee login lookup.
`GetAll`	`[dbo].[GetAllEmployees]`	Employee list.
`GetById`	`[dbo].[GetEmployeeById]`	Employee detail.
`Delete`	`[dbo].[DeleteEmployee]`	Employee delete.

Dapper Patterns

```
var employees = await connection.QueryAsync<EmployeeEntity>(
    "[dbo].[GetAllEmployees]",
    commandType: CommandType.StoredProcedure);
```

```
return await connection.QueryFirstOrDefaultAsync<EmployeeEntity>(
    "[dbo].[GetEmployeeById]",
    new { Id = id },
    commandType: CommandType.StoredProcedure);
```

```
var affectedRows = await connection.ExecuteScalarAsync<int>(
    "[dbo].[DeleteEmployee]",
    new { Id = id },
    commandType: CommandType.StoredProcedure);
```

Stored Procedure Examples

```
CREATE OR ALTER PROCEDURE [dbo].[GetEmployeeByEmail]
    @Email NVARCHAR(255)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Name, Email, PhoneNumber, PasswordHash, Role, Salary, DepartmentId, ClientI
    FROM Employees
    WHERE Email = @Email;
END
```

```
CREATE OR ALTER PROCEDURE [dbo].[GetAllEmployees]
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Name, Email, PhoneNumber, PasswordHash, Role, Salary, DepartmentId, ClientI
    FROM Employees
    ORDER BY Id DESC;
END
```

```
CREATE OR ALTER PROCEDURE [dbo].[GetEmployeeById]
    @Id INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Name, Email, PhoneNumber, PasswordHash, Role, Salary, DepartmentId, ClientI
    FROM Employees
    WHERE Id = @Id;
END
```

```
CREATE OR ALTER PROCEDURE [dbo].[DeleteEmployee]
    @Id INT
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM Employees
    WHERE Id = @Id;

    SELECT @@ROWCOUNT;
END
```

12. SMTP Email Sending

When a Manager creates an employee, the backend saves the employee and then sends an account-created email.

SMTP Config

Configuration is in `appsettings.json` under `MailSettings`.

```
"MailSettings": {
  "Enabled": true,
  "Host": "smtp.gmail.com",
  "Port": 587,
  "UseSsl": true,
  "UserName": "your-email@gmail.com",
  "Password": "your-app-password",
  "FromEmail": "your-email@gmail.com",
  "FromName": "Employee API"
}
```

> **Security note:** do not commit real SMTP passwords to a public repository. Prefer user secrets, environment variables, or deployment secret storage. If a real app password was committed or shared, rotate it.

MailSettings Class

`Configuration/MailSettings.cs` maps the JSON config into a C# object.

Email Sender Interface

`Service/Notifications/IEmployeeEmailSender.cs` defines:

```
Task SendAccountCreatedAsync(EmployeeEntity employee, string plainPassword);
```

SMTP Implementation

`Service/Notifications/SmtpEmployeeEmailSender.cs` uses `System.Net.Mail` to send the message.

Employee Create Flow With Email

```
var createdEmployee = await _service.Create(employee);
await _employeeEmailSender.SendAccountCreatedAsync(createdEmployee, model.Password);
```

The email uses `model.Password` because that is the original plain password. The database stores only `PasswordHash`.

13. Frontend Integration

The frontend should call:

```
POST /api/auth/login
```

Request body:

```
{
  "username": "employee@example.com",
  "password": "EmployeePassword"
}
```

Protected API calls must include:

```
Authorization: Bearer <token>
```

The frontend should decode the JWT role and show or hide pages based on:

- `Manager`: employee management, clients, departments, assigned tasks.
- `Employee`: employee-facing pages only.

14. Testing Checklist

1. Run database update if `PasswordHash` and `Role` are not in the database.
2. Confirm stored procedures exist in `EmployeeApiDb` under schema `dbo`.
3. Restart backend.
4. Login as manager using `admin/admin123`.
5. Use manager token to call Manager-only APIs.
6. Create employee with email, password, role, salary, department id, and optional client id.
7. Confirm employee row has `PasswordHash` and `Role`.
8. Confirm account email arrives if `MailSettings.Enabled` is true.
9. Login as employee using email/password.
10. Verify employee token cannot access Manager-only APIs.

15. Common Errors

Error	Cause	Fix
`Invalid username or password`	Email not found, wrong password, empty password hash.	Check employee email and `PasswordHash`.
`Could not find stored procedure`	SP missing, wrong database, wrong schema, wrong name.	Create procedure in `EmployeeApiDb` as `dbo`.
`401 Unauthorized`	Missing, invalid, or expired token.	Login again and send bearer token.
`403 Forbidden`	Token valid but role not allowed.	Use Manager token or change role/authorization rule.
`EmployeeApi.exe is being used`	Backend process is running during build.	Stop backend and rebuild.
Email not sending	SMTP disabled, wrong app password, wrong port/SSL, spam folder.	Check `MailSettings` and provider settings.

16. Current Design Notes

- The system currently mixes EF Core and Dapper. This is acceptable during migration/learning.
- `Create` and `Update` still use EF Core in `EmployeeService`.
- `GetByEmail`, `GetAll`, `GetById`, and `Delete` use Dapper stored procedures.
- SMTP email sending is synchronous in the request flow. If SMTP fails, employee creation response may fail after the employee is saved. A later improvement is to catch/log email errors or use a background queue.
- Role names must match exactly: `Manager` and `Employee`.

17. Recommended Next Improvements

1. Move real SMTP password out of `appsettings.json` into user secrets or environment variables.
2. Add a password reset/change password endpoint.
3. Restrict employee self-access by comparing `ClaimTypes.NameIdentifier` with route employee id.
4. Standardize controller authorization rules, especially whether department/client APIs should be Manager-only or visible to employees.
5. Remove unused dependencies from `AuthApiController`, such as unused DbContext injection if no longer needed.
6. Convert remaining EF methods to stored procedures only after current behavior is stable.
7. Add validation to guarantee role values are only `Manager` or `Employee`.

18. End-To-End Summary

```
Manager logs in
  -> JWT contains Manager role
  -> Manager creates employee
  -> Password is hashed into PasswordHash
  -> Employee receives email with credentials
  -> Employee logs in using email/password
  -> Auth uses Dapper SP GetEmployeeByEmail
  -> PasswordHasher verifies password
  -> JWT contains employee role
  -> Authorize attributes allow or block API access
```